

Prácticas XP del proyecto

Relación entre prácticas y valores

Las prácticas XP son la materialización concreta de los valores. Donde los valores definen el por qué, las prácticas definen el qué y el cómo. En este proyecto se aplican ocho prácticas durante la Fase 2 (1 enero 2027 – 22 abril 2027), distribuidas a lo largo de los 8 sprints de desarrollo.

XP no es una metodología de proceso que compite con Scrum: es la capa de ingeniería técnica que opera dentro de los sprints. Scrum gestiona el proceso; XP gestiona la técnica.

La práctica Sustainable Pace no se incluye porque no forma parte del contenido del curso.

Las ocho prácticas

1. Pair Programming

Qué es. Dos developers trabajan en el mismo problema al mismo tiempo, en la misma estación de trabajo o sesión compartida. Uno escribe código (driver); el otro revisa, piensa en el diseño y anticipa problemas (navigator). Los roles rotan con regularidad.

Cómo se aplica. El equipo de developers —Cruz, Chen, Guzmán— forma pares rotativos para las historias de mayor criticidad técnica. La rotación es deliberada: evitar que el par se convierta en asignación permanente y que el conocimiento técnico quede concentrado en una persona.

Épicas y sprints prioritarios.

- EP04-US01 (sprint 5, 8 puntos): algoritmo de ranking por afinidad. Es el núcleo del motor de recomendación y el código de mayor criticidad del proyecto. Requiere pair obligatorio.
- EP05-US01 (sprint 6, 8 puntos): ponderación por afinidad con resultado determinista para perfiles iguales. El determinismo del algoritmo necesita verificación en tiempo real por un segundo desarrollador.
- EP01 (historias de autenticación y datos personales, sprints 1-3): la seguridad de datos personales de estudiantes es sensible. Pair Programming reduce el riesgo de errores de implementación (R8, exposición de datos, probabilidad 0.20).

Efecto en productividad. Pair Programming reduce la productividad aparente individual (dos personas en una sola tarea) pero mejora la calidad del código, distribuye conocimiento técnico y reduce defectos post-merge. Este costo es visible y aceptado.

Mitigación de riesgos. Mitiga R5 (rotación de personal, probabilidad 0.20): cuando un developer queda no disponible, el otro miembro del par tiene contexto

técnico suficiente para continuar.

2. TDD (Test-Driven Development)

Qué es. Ciclo de desarrollo en tres pasos: primero se escribe la prueba que falla (rojo), luego se escribe el código mínimo que la hace pasar (verde), luego se mejora la estructura interna sin cambiar el comportamiento (refactor). El orden importa: la prueba siempre va antes que el código.

Cómo se aplica. TDD es obligatorio para la lógica de negocio crítica del proyecto. No es una recomendación: ninguna historia de EP04 o EP05 puede cerrarse sin pruebas unitarias que validen los casos principales del comportamiento esperado. El criterio del DoD XP-1 (TDD verde antes del merge) hace esto verificable y no negociable.

Épicas y sprints prioritarios.

- EP02 (sprint 2): reglas de guardado automático de la encuesta vocacional. El guardado automático tiene lógica de estado que debe probarse antes de implementarse.
- EP04 (sprint 5): scoring del perfil vocacional. El algoritmo de recomendación es el componente de mayor riesgo técnico del proyecto. Las pruebas unitarias definen el contrato de comportamiento del motor antes de escribir una línea de producción.
- EP05 (sprint 6): determinismo del ranking. Para perfiles idénticos, el sistema debe producir el mismo orden de resultados siempre. TDD es la única forma confiable de garantizar esa propiedad.

Cobertura. La cobertura aplica sobre lógica de negocio crítica. Las métricas de cobertura por línea sobre todo el código base están fuera del alcance del curso y no se adoptan como objetivo.

Relación con Simple Design. Las pruebas TDD definen exactamente qué debe hacer el código. Esto hace que Simple Design no sea arbitrario: el código más simple es el que hace pasar las pruebas, sin más.

3. Refactoring

Qué es. Mejora continua de la estructura interna del código sin cambiar su comportamiento externo. El refactoring no añade funcionalidad ni corrige errores: reorganiza el código para que sea más legible, menos duplicado y más fácil de modificar.

Cómo se aplica. El refactoring es parte del DoD (criterio XP-3: sin deuda técnica deliberada), no una tarea separada ni un sprint dedicado al final del proyecto. Reservar aproximadamente el 20 % de la capacidad de cada sprint para historias técnicas de refactoring (incluidas en el backlog como HT-04 y HT-06) es la forma de hacer ese compromiso planificable y visible.

El anti-patrón explícito que el equipo evita es “refactor como tarea aparte”: acumular deuda técnica a lo largo de los sprints con la intención de limpiarla en un sprint final. Ese sprint final generalmente no existe o no tiene capacidad suficiente.

Épicas y sprints prioritarios.

- Sprint 3 (HT-04): refactorización del módulo de autenticación y registro (EP01), después de que la implementación inicial en sprints 1-2 ha madurado.
- Sprint 6 (HT-06): refactorización del motor de recomendación (EP04) una vez que la implementación del sprint 5 está estabilizada. Los sprints 5-7 concentran la mayor complejidad técnica del proyecto (EP04, EP05, EP06).

Relación con TDD. El refactoring seguro requiere pruebas existentes. Si el ciclo TDD se aplicó correctamente en la implementación original, el equipo puede refactorizar con confianza porque las pruebas detectan regresiones inmediatamente.

4. Continuous Integration (CI)

Qué es. Práctica de integrar el código al repositorio principal con frecuencia (al menos una vez por día) y ejecutar automáticamente un pipeline que compila el código, ejecuta las pruebas y reporta el estado. El pipeline es la red de seguridad que detecta problemas de integración antes de que se acumulen.

Cómo se aplica. El pipeline CI se establece en el sprint 1 (HT-01). Configurar CI tardíamente es un anti-patrón explícito del docente y del equipo: un pipeline que se activa en el sprint 7 no ha protegido los sprints anteriores.

El pipeline mínimo del proyecto tiene tres etapas: (1) compilación, (2) pruebas unitarias TDD, (3) reporte de cobertura de la lógica de negocio crítica.

La regla es operacional: ninguna historia puede marcarse como lista para revisión del PO si el pipeline CI reporta falla. Esta regla hace el DoD XP-2 verificable sin depender del juicio individual.

Épicas y sprints prioritarios. CI aplica a todos los sprints desde el primero. Su impacto es más visible en los sprints donde la integración entre componentes es más compleja:

- Sprint 5 (EP04): el motor de recomendación integra servicios cloud. Los problemas de integración cloud se detectan en horas, no en semanas.
- Sprints 6-7 (EP05, EP06): el panel de resultados integra el motor y el catálogo de carreras.

Mitigación de riesgos. Mitiga R2 (retraso técnico cloud, probabilidad 0.40): la integración con servicios cloud se verifica automáticamente en cada ciclo de

CI. Los problemas de conectividad, autenticación o formato de respuesta se detectan temprano.

5. Small Releases

Qué es. Entregas pequeñas y frecuentes de software funcionando. Cada entrega es un incremento real del sistema, no una demo de pantallas ni prototipos. La frecuencia de las entregas reduce el riesgo de construir algo incorrecto durante semanas antes de recibir retroalimentación real.

Cómo se aplica. La estructura de 8 sprints con entregables incrementales por épica es la materialización de Small Releases en este proyecto. Cada sprint cierra con un incremento potencialmente entregable:

Sprint	Entregable incremental
S1	EP01: registro de usuario y perfil básico
S2	EP01-EP02: encuesta vocacional con guardado automático
S3	EP02-EP03: catálogo de carreras pensum 2026
S4	EP03: gestión completa de carreras
S5	EP04: motor de recomendación funcional
S6	EP05: clasificación de afinidad y ponderación
S7	EP05-EP06: panel de resultados inicial
S8	EP06: panel completo con justificaciones, consolidación y estabilización final

Relación con el Sprint Review. Small Releases y el Sprint Review de Scrum son coherentes: el Review es el mecanismo formal de presentación del incremento. XP refuerza que ese incremento sea software funcionando, no una demo parcial.

6. Simple Design

Qué es. La solución más simple que cumple la historia de usuario es la correcta. Simple Design rechaza la infraestructura especulativa: no se construye para casos de uso futuros no confirmados, no se generaliza más allá del alcance definido, no se añaden capas de abstracción que la historia no requiere.

Cómo se aplica. Simple Design es la práctica que contiene la complejidad técnica del proyecto dentro del alcance real.

El caso más importante es EP04: el motor de recomendación está diseñado exclusivamente para el pensum 2026 de UVG Campus Central. No es un motor generalizable para otras universidades, otros años académicos o futuros cambios de pensum. Diseñarlo como si fuera reutilizable añadiría complejidad que el proyecto no puede absorber en 8 sprints.

EP05 aplica el mismo principio: el modelo de ponderación es el más simple que produce resultados deterministas para perfiles iguales. No se añaden capas de ponderación dinámica, personalización post-resultado ni mecanismos de retroalimentación del usuario sobre el ranking.

Relación con TDD. TDD y Simple Design se refuerzan mutuamente. Las pruebas definen el contrato exacto del código; Simple Design garantiza que ese contrato no se extiende más allá de lo necesario.

Anti-patrón. El riesgo de Simple Design es confundirlo con descuido. Simple no significa mal estructurado: significa que la estructura del código es proporcional a la complejidad del problema que resuelve.

7. Collective Ownership

Qué es. Cualquier developer del equipo puede leer, entender y modificar cualquier parte del código base en cualquier momento. No hay módulos que “le pertenezcan” a un developer específico de forma exclusiva y permanente.

Cómo se aplica. Los tres developers —Cruz, Chen, Guzmán— no tienen asignaciones exclusivas de módulo. Durante el sprint, trabajan sobre las historias priorizadas independientemente de quién escribió el código previo.

La condición necesaria para Collective Ownership son los estándares de codificación acordados. HT-02 (sprint 1) define y documenta esos estándares antes de que comience el desarrollo de funcionalidad. Sin estándares comunes, “cualquiera puede modificar cualquier parte” genera inconsistencia en lugar de flexibilidad.

Épicas y sprints prioritarios. Collective Ownership aplica a todo el código base, pero su importancia es máxima en los sprints donde la complejidad es alta (5-7) y en los módulos de seguridad (EP01, EP02), donde el conocimiento sobre manejo de datos personales no puede quedar en una sola persona.

Mitigación de riesgos. Mitiga R5 (rotación de personal, probabilidad 0.20): si un developer queda no disponible, los otros dos tienen contexto técnico suficiente sobre todos los módulos para continuar el desarrollo sin pérdida catastrófica de conocimiento.

Relación con Pair Programming. Pair Programming construye Collective Ownership activamente: al rotar pares, cada developer desarrolla familiaridad con código que no escribió originalmente. Collective Ownership es el resultado acumulado de Pair Programming a lo largo de los sprints.

8. Nota sobre la columna Code Review en el tablero

Cuando se incorporan prácticas XP al tablero Kanban/Scrumban del proyecto, se agrega una columna “Code Review” entre Testing y Review del PO. Esta columna no es una práctica XP per se: es el efecto visible de Pair Programming en el flujo de trabajo del tablero. La columna materializa el paso de revisión técnica por pares y hace visible un control de calidad que sin XP quedaría implícito o se omitiría bajo presión de velocidad.

El detalle completo de esta columna y su efecto en el Cycle Time se documenta en `nota-columna-tablero.md`.

Resumen: prácticas y su vínculo con el proyecto

Práctica	Sprint de activación	Épica principal	Riesgo mitigado
Pair Programming	Sprint 1	EP01, EP04, EP05	R5, R8
TDD	Sprint 1	EP02, EP04, EP05	—
Refactoring	Sprint 3	EP01, EP04	—
Continuous Integration	Sprint 1	Todos	R2
Small Releases	Sprint 1	Todos	—
Simple Design	Sprint 1	EP04, EP05	—
Collective Ownership	Sprint 1	Todos	R5